基于 ReL4 微内核的组件化重构与形式化验证研究开题报告

suspen

2026 - 03 - 09

Contents

1 课题背景与意义 1

2 国内外研究现状 2

3 研究目标与内容 2
- 3.1 研究目标 2
- 3.2 研究内容 2

4 拟解决的关键问题 3

5 研究方法与技术路线 4
- 5.1 研究方法 4
- 5.2 技术路线 4

6 预期成果 4

7 进度安排 5

8 参考文献 5

1 课题背景与意义

微内核架构因其高安全性、良好可扩展性而备受关注. 前期工作已使用 Rust 语言重写了 seL4 微内核，形成 ReL4，并在其上移植了 unikernel ArceOS，为微内核的性能优化和功能扩展奠定了基础.

目前 ReL4 社区已将内核按功能拆分为多个独立仓库（如 `sel4_task`、`sel4_ipc`、`sel4_vspace`、`sel4_cspace` 等），实现了初步的模块化. 然而，现有模块化仍以功能划分为主，模块间的接口定义尚不统一，部分模块内部耦合度较高，且与 seL4 特有的能力系统深度绑定. 这些因素限制了模块的独立复用和灵活配置，也增加了形式化验证的复杂度.

因此，有必要在现有模块化基础上进行进一步的组件化探索：借鉴 ArceOS 等组件化操作系统的分层设计思想，结合微内核自身特点，重新梳理模块边界，定义清晰的接口契约，逐步将内核功能拆分为粒度更细、可复用性更高的组件. 同时，借助 Rust 语言的内存安全特性和新兴验证工具（如 Verus），对关键组件进行形式化验证，探索构建高可信微内核的新路径.

本课题旨在提升 ReL4 的可维护性、可配置性和可验证性，为后续微内核系统的定制化开发和可信保障提供技术支撑.

2 国内外研究现状

1. **微内核与 seL4**：seL4 是经过形式化验证的高性能微内核，其核心设计包括能力空间（CSpace）、虚拟地址空间（VSpace）、任务控制块（TCB）和 IPC 等机制，所有内核操作均通过能力授权. ReL4 是 seL4 的 Rust 重写版本，继承了其架构并利用 Rust 语言的安全性强化了实现.
2. **组件化操作系统**：Unikraft、MirageOS 等项目展示了通过组件化构建专用操作系统的可行性. ArceOS 作为 Rust 编写的 unikernel，通过分层架构（如 `axalloc`、`axtask`、`axhal`）实现了高度模块化，各模块通过 trait 定义接口，支持可插拔实现. 这些设计为微内核的组件化提供了借鉴.
3. **Rust 与形式化验证**：Rust 的所有权系统在编译时保证内存安全和无数据竞争，消除了 C 语言中常见的内存错误. Verus 是专为 Rust 设计的验证工具，能够利用 Rust 类型信息自动合成证明，大幅降低验证成本. AI 辅助证明工具（如 AlphaVerus）则进一步展示了自动化验证的潜力.
4. **当前 ReL4 模块化现状**：ReL4 已分解为多个独立 crate，但模块间仍存在具体类型依赖，与能力系统紧耦合，尚未实现真正的可复用抽象. 如何在不破坏 seL4 核心设计的前提下，通过合理的接口抽象提升组件独立性，是当前研究的重要方向.

3 研究目标与内容

3.1 研究目标

1. 深入分析 ReL4 现有模块结构，识别可进一步组件化的功能点，制定合理的组件划分方案.
2. 借鉴 ArceOS 等系统的分层设计，结合微内核特点，探索构建硬件抽象层（HAL）、策略组件（如调度器、内存分配器）和核心机制（如 IPC、能力空间）之间的清晰边界.
3. 通过迭代重构，逐步将 ReL4 改造为一组接口清晰、可独立复用、可配置的内核组件，并尝试发布到 crates.io.
4. 利用用户态中断等软硬协同技术，优化微内核关键路径性能.
5. 结合 Rust 语言安全特性和 Verus 工具，对部分关键组件进行形式化验证，量化验证收益，总结验证方法.

3.2 研究内容

3.2.a 现有模块代码梳理与评估

- 分析现有 `sel4_*` 模块的代码结构、依赖关系和 unsafe 代码分布.
- 修复警告，规范命名，收缩 unsafe 范围，为后续重构奠定基础.
- 识别与能力系统耦合紧密的部分和可能解耦的策略部分，明确重构的优先级和可行性.

3.2.b 组件接口设计与标准化

- 参考 ArceOS 的 `axhal`、`axalloc` 等模块，设计硬件抽象层接口（控制台、定时器、中断等），实现架构无关的硬件操作.
- 分析调度器、物理内存分配器等策略性功能，定义通用 trait，使不同算法可插拔.
- 对于 IPC、能力空间等核心机制，评估内部进一步拆分的合理性，保持其整体性的同时优化内部结构.

3.2.c 核心机制组件化实现（思路一）

- 对于与能力系统深度绑定的模块（如 CSpace、IPC 核心），保持其整体性，但通过内部模块化优化代码组织，减少内部耦合.
- 明确这些模块对外的依赖关系，尽可能将架构相关代码（如页表操作、上下文切换）剥离到 HAL 中.

3.2.d 策略组件抽象与实现（思路二）

- **调度器抽象**：定义 `Scheduler` trait，实现多种算法（如 FIFO、RR、CFS）作为独立 crate，供任务管理模块调用.
- **物理内存分配器抽象**：定义 `PhysFrameAlloc` trait，实现伙伴系统、位图分配器等，供 VSpace 模块使用.
- **页表操作抽象**：定义 `PageTable` trait，为不同架构提供独立实现，使 VSpace 模块与架构解耦.

3.2.e 硬件抽象层（HAL）构建

- 将控制台、定时器、中断控制器等硬件操作抽象为 trait，为 RISC-V、x86_64 等架构提供具体实现.
- 整合现有 `serial-*`、`driver-collect`，形成统一的驱动框架.

3.2.f 性能优化探索

- 分析组件化后的 IPC 开销，调研用户态中断等新技术，尝试优化 IPC 路径.
- 通过基准测试评估优化效果.

3.2.g 形式化验证探索

- 分析 Rust 语言安全特性（所有权、借用检查）覆盖的内存安全属性，与 seL4 的 C 语言验证进行对比，量化 Rust 带来的验证收益.
- 选取关键组件（如调度器、能力模型）使用 Verus 编写规范和证明，记录验证工作量.
- 探索 AI 辅助证明工具（AlphaVerus）在自动生成证明方面的潜力.

4 拟解决的关键问题

1. **组件划分的合理性**：如何在不破坏 seL4 核心设计的前提下，识别可独立复用的功能点，定义合适的接口粒度.
2. **与能力系统的解耦程度**：哪些模块可以真正与能力系统解耦？哪些必须保持紧密耦合？需通过实际代码分析确定.
3. **性能开销控制**：接口抽象可能引入动态分派开销，需评估并优先使用静态分派（泛型）保持零成本抽象.
4. **多架构支持**：HAL 和页表操作需为多种架构提供统一接口，同时允许架构优化.
5. **形式化验证的成本效益**：如何高效利用 Verus 证明关键性质，并与 Rust 语言自身保证相结合.

5 研究方法与技术路线

5.1 研究方法

- **文献研究**：深入研读 seL4、ReL4、ArceOS、Unikraft 等相关文献，借鉴设计思想.
- **代码分析**：通过静态分析、依赖图构建等方法，理解现有模块间的耦合关系.
- **迭代式重构**：采用小步快跑的方式，逐步进行组件拆分和接口抽象，每一步通过测试保证功能正确性.
- **测试驱动开发**：编写单元测试和集成测试，确保重构后系统行为不变.
- **性能分析**：使用 perf、flamegraph 等工具分析性能瓶颈.
- **形式化验证**：学习 Verus 工具，对关键模块建模和证明；尝试 AI 辅助证明工具.

5.2 技术路线

1. **代码梳理阶段**：修复警告，规范命名，收缩 unsafe，建立代码质量基准.
2. **依赖分析阶段**：分析各模块间的具体依赖，识别紧耦合点和潜在抽象点.
3. **HAL 构建阶段**：先实现架构无关的硬件抽象，验证可行性.
4. **策略组件抽象阶段**：逐步提取调度器、内存分配器接口，并实现多种算法作为独立 crate.
5. **核心机制优化阶段**：对 IPC、能力空间等模块进行内部重构，优化结构但不破坏整体性.
6. **集成与测试阶段**：将重构后的组件重新组合成可运行的 ReL4 内核，运行 ArceOS 验证功能正确性.
7. **性能优化阶段**：分析瓶颈，引入用户态中断等技术优化.
8. **形式化验证阶段**：选取 1-2 个关键组件进行 Verus 验证，记录数据.
9. **文档与发布阶段**：编写详细文档，尝试发布组件到 crates.io.

6 预期成果

1. **组件化重构后的 ReL4 内核**：代码结构清晰，模块边界明确，unsafe 代码范围大幅缩减.
2. **一组可复用的内核组件**：
   - 硬件抽象层（HAL）及多架构实现
   - 调度器 trait 及多种算法实现
   - 物理内存分配器 trait 及多种算法实现
   - 页表操作 trait 及多架构实现
   - 核心机制模块（CSpace、IPC）内部优化版本
3. **性能优化报告**：用户态中断等技术的优化效果分析.
4. **形式化验证成果**：
   - Rust 语言安全特性减少验证工作量的量化分析
   - 至少 1 个关键组件的 Verus 验证实现
   - AI 辅助证明工具的探索实验报告
5. **毕业设计论文一篇**：详细阐述组件化重构的设计、实现与验证过程.
6. **开源代码仓库**：所有代码托管于 GitHub，附带完整文档和示例.

7 进度安排

阶段	时间	主要工作内容
阶段一：准备与调研	第 1-4 周	熟悉 ReL4 模块仓库，搭建开发环境；梳理代码，修复警告，收缩 unsafe；分析模块依赖关系，明确组件化方案.
阶段二：核心组件化重构	第 5-10 周	构建硬件抽象层（HAL）并为 RISC-V/ARM 实现；逐步抽象调度器、内存分配器接口并实现示例算法；对 CSpace/IPC 等核心机制进行内部优化.
阶段三：集成与优化	第 11-13 周	将重构组件组合运行，验证功能正确性；调研用户态中断等技术，评估并优化 IPC 性能.
阶段四：形式化验证与论文撰写	第 14-16 周	选取关键组件进行 Verus 验证，探索 AI 辅助证明；整理研究成果，完成毕业设计论文初稿，修改并准备答辩.

8 参考文献

[1] Klein G, et al. seL4: Formal verification of an OS kernel. ACM Symposium on Operating Systems Principles, 2009.

[2] seL4 Foundation. seL4 Reference Manual.

[4] ArceOS Team. ArceOS: An Unikernel Written in Rust. https://github.com/arceos-org/arceos

[5] Lattuada A, et al. Verus: A Practical Foundation for Systems Verification. SOSP '24, 2024.

[6] Unikraft. Unikraft: Fast, Specialized Unikernels the Easy Way.

[7] Madhavapeddy A, et al. MirageOS: A Library Operating System for Unikernels.

[8] Verus Language Project. https://verus-lang.github.io/verus/guide/

[9] AlphaVerus: Bootstrapping Formally Verified Code Generation through Self-Improving Translation and Treefinement. ICML 2025.

[10] LionsOS: A Verified Operating System for Highly Assured Systems. (In submission)